PROJECT PROGRESS REPORT

on

Failure-Aware Fast Rerouting in Software-Defined Networks

Interim review

Submitted by

S. No.	Name of the Student	Roll Number
1.	Mayank Bansal	23BEC026
2.	Anas Khan	23BCS083
3.	Shardendu Mishra	23BCS122
4.	Himanshu Kumar	23BCS055
5.	Divyanshu	23BEC017

Under the guidance of

_____

(Name and designation of the guide)

Department of _____

_____

(Name of the Institute / University)

Date of review: _____

1. Introduction

When a link or switch fails in a Software-Defined Network (SDN), traffic stops until forwarding is repaired. If the repair goes through the controller, the switch must report the failure, the controller must recompute paths and every affected switch must be reprogrammed, which takes tens to hundreds of milliseconds. OpenFlow 1.3 offers *fast-failover groups*, which let a switch hold a pre-computed backup and switch to it locally. This project studies fast rerouting in SDN and aims to make it **failure-aware**: backups that take into account how likely links are to fail and which links fail together.

This report describes the progress made so far: the literature reviewed, the plan, the emulator built for the experiments, the recovery strategies implemented and the failure detection mechanisms added to it, and the first results.

2. Literature Review (Summary)

- **Restoration vs protection.** Sharma et al. [2] measured OpenFlow restoration (controller recomputes) against protection (backup pre-installed); only protection meets the 50 ms carrier-grade target on larger networks.
- **Fast-failover groups.** van Adrichem et al. [3] showed that with OpenFlow fast-failover groups and BFD, recovery time is dominated by failure detection.
- **IP/MPLS fast reroute.** Loop-Free Alternates [4] and MPLS fast reroute [5] protect next hops and nodes with local detours; this project borrows their link and node protection model.
- **Failure detection.** Fast reroute can only react to failures that are detected. Loss of signal is immediate but misses *gray* failures, where a link stays up but drops traffic; BFD [7] exchanges hellos every few milliseconds and catches both; controller-driven LLDP probing also catches both, but slowly.
- **Disjoint paths.** Suurballe/Bhandari algorithms [6] compute link- or node-disjoint path pairs for end-to-end protection.
- **Gap.** These schemes treat all links as equally reliable and plan each backup independently. Links that share a conduit (a shared risk link group, SRLG) can fail together, which can take out a primary and its backup at the same time.

3. Problem Statement and Objectives

Problem. Design and evaluate a fast-reroute scheme for OpenFlow networks that recovers in milliseconds, like data-plane protection, but whose backups remain valid under correlated and repeated failures.

1. Build a deterministic OpenFlow network emulator that models flow and group tables, port liveness and control-channel delay.
2. Implement baseline recovery strategies: no recovery, controller-based restoration, and local fast reroute with link and node protection.
3. Model failure detection (loss of signal, BFD, LLDP) and realistic failure types: link cuts, gray (silent) failures and switch crashes.
4. Measure recovery time, packet loss and switch state on research backbones.
5. Design and implement a failure-aware fast-reroute scheme (risk model, SRLG-aware backups) and compare it with the baselines.
6. Port the scheme to a real OpenFlow controller for Mininet and Open vSwitch.

4. Methodology and Plan

The system is developed in versions, each adding one capability with its own tests, from the emulator to the proposed scheme (Figure 4.1, Table 4.1).

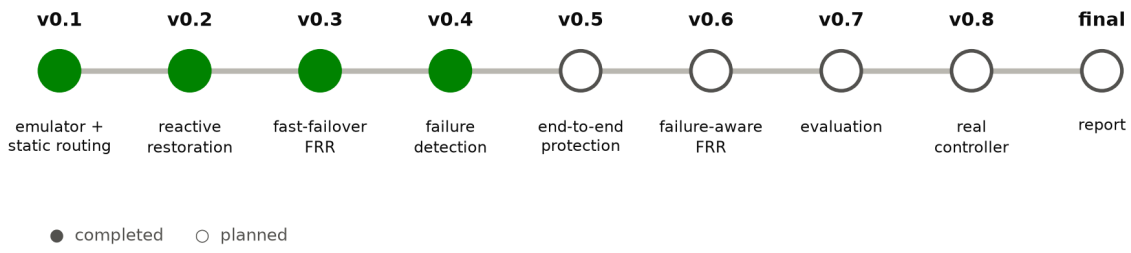


Figure 4.1: Development plan; the first four versions are complete.

Table 4.1: Versions and status

Version	Content	Status
v0.1	discrete-event emulator, OpenFlow flow tables, static shortest-path routing	done
v0.2	PORT_STATUS, reactive restoration by the controller	done
v0.3	group tables, fast-failover, tagged link and node detours	done
v0.4	failure detection: loss of signal, BFD, LLDP; gray failures, switch crashes	done
v0.5	end-to-end path protection with disjoint paths	next
v0.6	failure-aware FRR: risk model, SRLG-aware backups, re-optimisation	planned
v0.7	evaluation suite on Abilene and NSFNET	planned
v0.8	port to OS-Ken/Ryu for Mininet and Open vSwitch	planned

5. Work Completed

5.1 Emulator

A discrete-event OpenFlow emulator was written in Python (about 2,100 lines, 78 automated tests). It models switches with prioritised flow tables and group tables (ALL, INDIRECT, FAST_FAILOVER), links with propagation delay, ports with liveness, hosts, and a controller connected through a control channel with one-way delay, computation time and per-rule installation time (Figure 5.1). Constant-bit-rate flows carry sequence numbers, so every lost packet is counted and the outage of a flow is the longest run of consecutive losses.

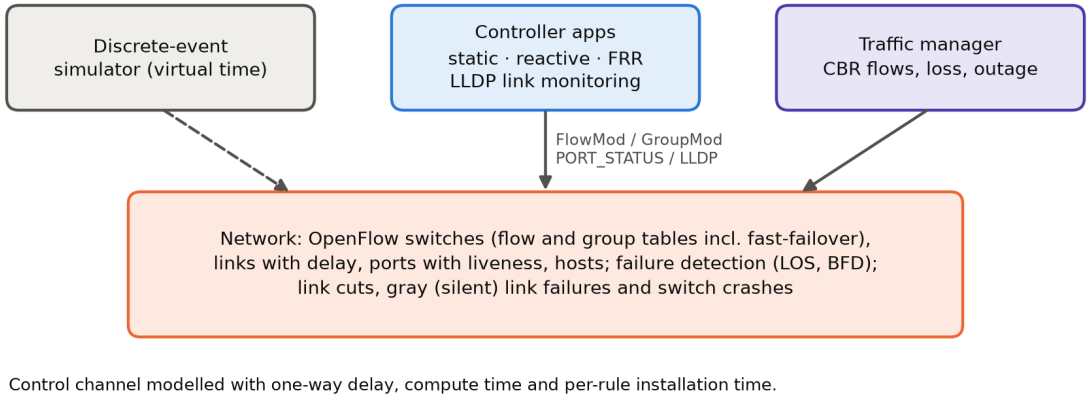


Figure 5.1: Emulator architecture.

5.2 Recovery Strategies Implemented

Table 5.1: Strategies implemented so far

Strategy	How it recovers
static	shortest-path routing, no recovery (baseline)
reactive	on PORT_STATUS the controller recomputes all paths and pushes the changed rules
frr-link	every switch holds a fast-failover group per destination: primary next hop, plus a VLAN-tagged detour that avoids the failed link
frr-node	as frr-link, but the detour avoids the whole next switch, so it also covers switch failures

Detours are planned on the shortest-path tree of the network without the protected element, so one tag per protected element is enough and the detours are loop-free.

5.3 Failure Detection and Failure Types

Port liveness, which fast-failover groups act on, now comes from pluggable detectors (Table 5.2), and the emulator can inject three kinds of failure (Table 5.3).

Table 5.2: Failure detection mechanisms

Mechanism	Who detects	How	Gray failures
loss of signal (LOS)	switch	the physical signal disappears (about 1 ms)	no
BFD	switch	hellos every interval on every link; down after 3 missed	yes
LLDP probing	controller	probes sent through every link; down after 3 missed rounds, then PORT_STATUS-like handling	yes

Table 5.3: Failure types

Failure	What happens
link cut	the link drops everything and its signal disappears, so LOS sees it
gray (silent) failure	the link drops everything but its signal stays up; only BFD or LLDP can see it
switch crash	the switch stops forwarding and all of its links go dark

Every probe packet is counted, so the cost of faster detection can be compared with the outage it saves.

6. Preliminary Results

Setup: Abilene (11 switches, 14 links, delays from fibre distance) and NSFNET (14 switches, 21 links), one host per switch, traffic between every pair of hosts at 500 packets per second, controller one-way delay 5 ms. Each link is cut in turn at t = 0.4 s.

Table 6.1: Single link failures: mean outage of affected flows over all links

Strategy	Abilene outage	NSFNET outage	Lost pkts (Abilene)	Flow / group entries
static	never	never	4,120	121 / 0
reactive	23.3 ms	20.6 ms	457	121 / 0
FRR (link)	5.8 ms	3.0 ms	107	440 / 110
FRR (node)	5.8 ms	3.0 ms	107	404 / 110

Fast reroute recovers about four times faster than reactive restoration (5.8 ms against 23.3 ms on Abilene), at the cost of about 3.5 times more flow entries plus one group per switch and destination. Its recovery time does not depend on how far away the controller is, while reactive restoration grows with every millisecond of control-channel delay (Figure 6.1).

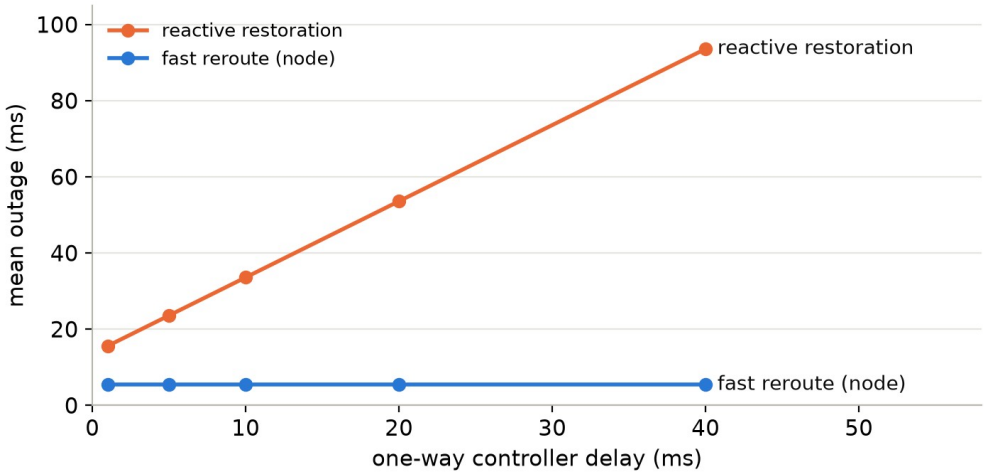


Figure 6.1: Mean outage after the DEN–KSC cut on Abilene as the controller moves further away.

Gray failures. DEN–KSC is made to drop every packet while its signal stays up (Table 6.2). With loss of signal alone the failure is never noticed and all 46 flows crossing the link stay down. BFD detects it, and its interval sets a trade-off between outage and probe traffic; LLDP probing by the controller also detects it but is about slower than BFD for the same number of probes, and the repair still has to go through the controller.

Table 6.2: Gray failure of DEN–KSC on Abilene (46 affected flows)

Detection	Mean outage	Lost packets	Probe packets
FRR (node), loss of signal only	never	9,651	0
FRR (node), BFD 1 ms × 3	6.6 ms	302	19,600
FRR (node), BFD 3.3 ms × 3	12.3 ms	565	5,937
FRR (node), BFD 10 ms × 3	31.1 ms	1,432	1,960
FRR (node), BFD 50 ms × 3	144.3 ms	6,637	392
reactive, LLDP 50 ms × 3	167.6 ms	7,711	392

Switch crashes. When the KSC switch crashes, 62 flows are affected; the 20 flows that start or end at KSC cannot be recovered by any scheme (Table 6.3). Node protection recovers all the others in 9.1 ms, while link protection leaves 19 of them down, because their detours pass through the crashed switch.

Table 6.3: Crash of switch KSC on Abilene

Strategy	Flows never recovered	Mean outage (recovered)	Lost packets
static	62 / 62	–	13,017
reactive	20 / 62	27.3 ms	5,266
FRR (link)	39 / 62	9.1 ms	8,367
FRR (node)	20 / 62	9.1 ms	4,500

7. Observed Limitation

The two Abilene links DEN–KSC and SNV–DEN are assumed to share a conduit. When both fail at the same instant, the detour that fast reroute pre-installed for one of them often uses the other (Table 7.1).

Table 7.1: DEN–KSC and SNV–DEN fail together

Strategy	Affected flows	Flows never recovered	Lost packets
static	50	50	10,549
reactive	50	0	1,339
FRR (link)	50	27	5,888
FRR (node)	50	22	4,887

Reactive restoration recovers every flow, slowly; fast reroute recovers most flows quickly but leaves 22–27 flows permanently disconnected, because the controller never repairs a backup that has failed. This is the gap the remaining work addresses.

8. Remaining Work

- **End-to-end protection (v0.5):** disjoint primary/backup paths as a further baseline.
- **Failure-aware fast reroute (v0.6):** a per-link risk model learned from observed failures, backups that avoid links sharing a risk group with the primary, and background re-optimisation after each failure so the network is protected again.
- **Evaluation (v0.7):** single, correlated, sequential and flapping failures on Abilene and NSFNET.
- **Real controller (v0.8):** OS-Ken/Ryu application for Mininet and Open vSwitch.
- **Final report and presentation.**

References

[1] N. McKeown et al., “OpenFlow: enabling innovation in campus networks,” ACM SIGCOMM CCR, vol. 38, no. 2, 2008.

[2] S. Sharma, D. Staessens, D. Colle, M. Pickavet, P. Demeester, “OpenFlow: meeting carrier-grade recovery requirements,” Computer Communications, vol. 36, no. 6, 2013.

[3] N. L. M. van Adrichem, B. J. van Asten, F. A. Kuipers, “Fast recovery in software-defined networks,” EWSDN, 2014.

[4] A. Atlas, A. Zinin, “Basic specification for IP fast reroute: loop-free alternates,” RFC 5286, 2008.

[5] P. Pan, G. Swallow, A. Atlas, “Fast reroute extensions to RSVP-TE for LSP tunnels,” RFC 4090, 2005.

[6] R. Bhandari, Survivable Networks: Algorithms for Diverse Routing, Kluwer, 1999.

[7] D. Katz, D. Ward, “Bidirectional Forwarding Detection (BFD),” RFC 5880, 2010.

[8] S. Knight et al., “The Internet Topology Zoo,” IEEE JSAC, vol. 29, no. 9, 2011.